

Main tasks:

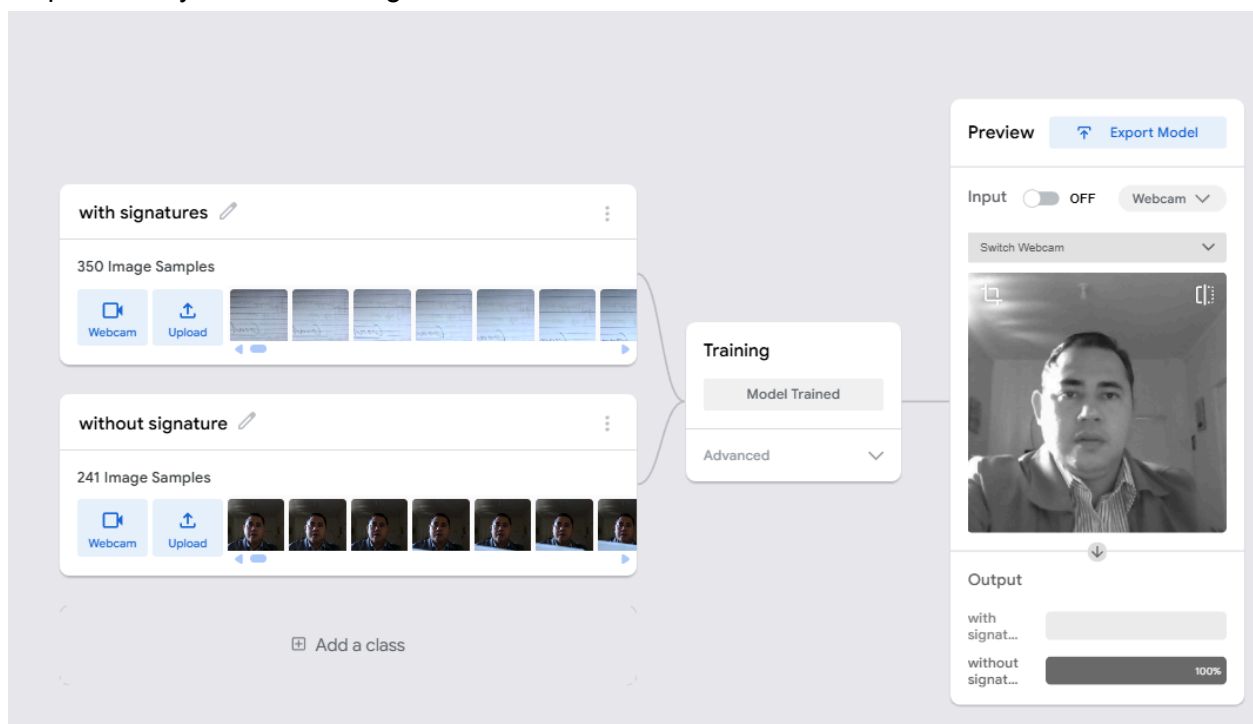
Create simple google form where respondents will upload a signed/ unsigned document/ of valid or not valid document

Then make sure you have your folder of those document on your google drive, then start training your model

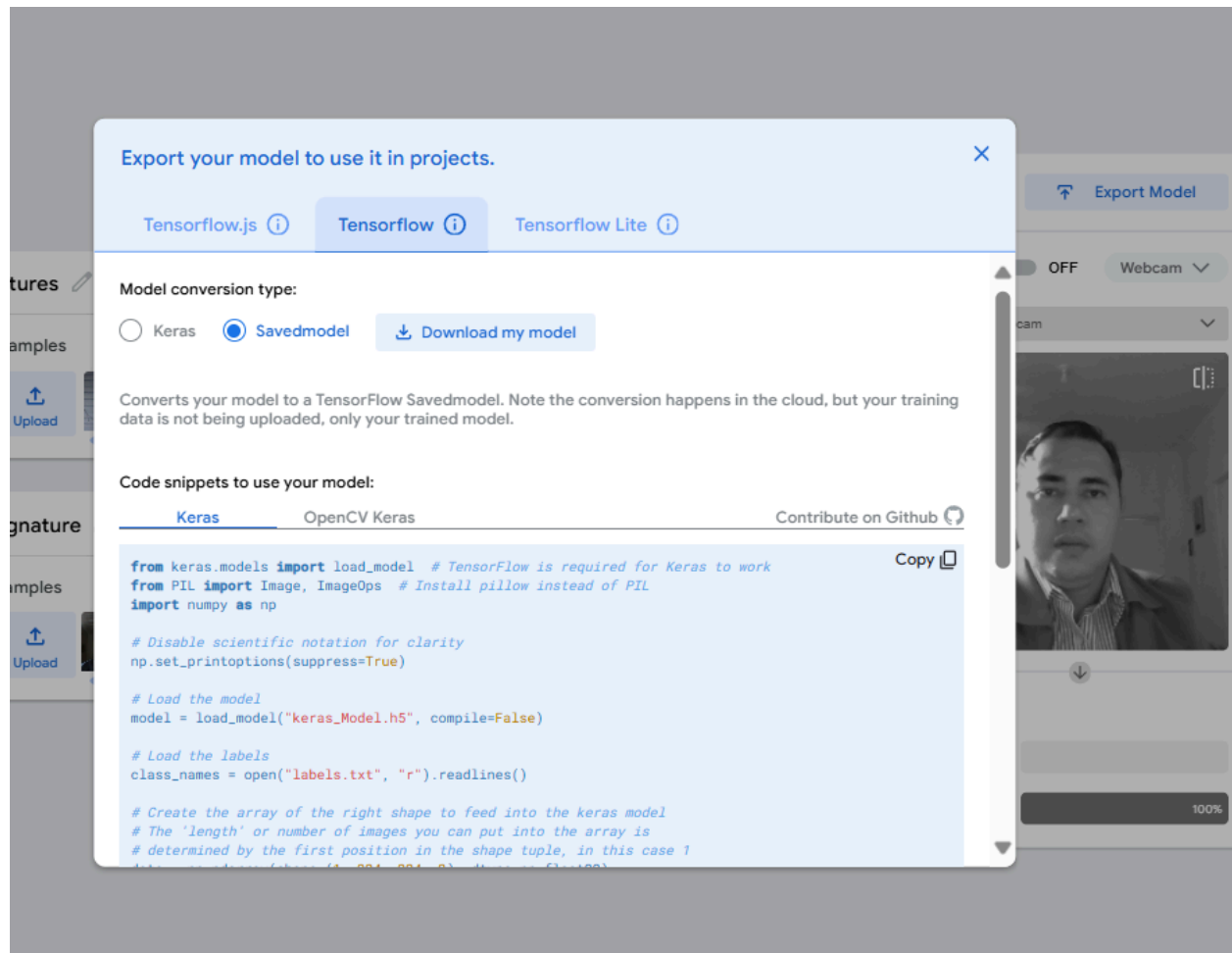
Step 1:

Visit <https://teachablemachine.withgoogle.com/>

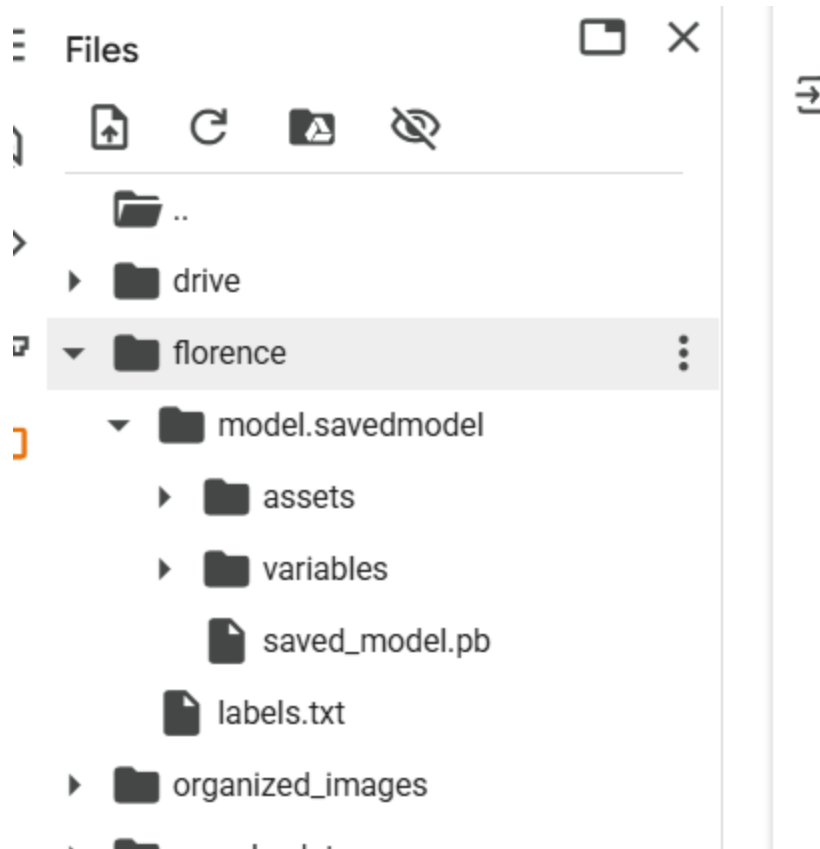
Step 2: Train your model using video



Step 3: export your model



Step 4: in Google Colab, create your directory of your savedmodel



Step 5: Then apply the following steps

Step 1:

```
import tensorflow as tf
import json
import os
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
from google.colab import drive
import glob
import pandas as pd
from datetime import datetime
import shutil
```

Step 2:

```
# ===== MOUNT GOOGLE DRIVE =====  
print("🔗 Mounting Google Drive...")  
drive.mount('/content/drive')  
print("✅ Google Drive mounted successfully!")  
  
# ===== MODEL LOADING SECTION =====  
model_path = '/content/florence/model.savedmodel' # Change this to your  
actual path  
  
try:  
    model = tf.saved_model.load(model_path)  
    predict_fn = model.signatures['serving_default']  
    print("✅ SUCCESS! Model loaded as SavedModel")  
except Exception as e:  
    print(f"❌ Error loading SavedModel: {e}")  
    raise
```

Step 3:

```
# Clean and parse class names properly
labels_path = '/content/florence/labels.txt'
try:
    with open(labels_path, 'r') as f:
        raw_labels = [line.strip() for line in f.readlines()]

    # Parse the labels correctly (handling "0 with signatures", "1 without
    signatures")
    class_names = []
    for label in raw_labels:
        if 'with' in label.lower() and 'signature' in label.lower():
            if 'without' in label.lower():
                class_names.append('without signature')
            else:
                class_names.append('with signature')
        else:
            # Fallback parsing
            class_names.append(label)

    print(f"✅ Parsed {len(class_names)} classes:")
    for i, name in enumerate(class_names):
        print(f"    {i}: {name}")

except Exception as e:
    print(f"⚠️ Using default class names due to error: {e}")
    class_names = ['with signature', 'without signature']

# Class emojis for better visualization
class_emojis = {
    'with signature': '✍️',
    'without signature': '📄'
}
```

Step 4:

```
# ===== PREDICTION FUNCTIONS =====
def predict_image(image_path, show_visualization=False):
    """
    Predict if an image has a signature or not
    """
    try:
        # Load and preprocess image
        img = Image.open(image_path)
        original_img = img.copy()

        # Convert to RGB if necessary
        if img.mode == 'RGBA':
            rgb_img = Image.new('RGB', img.size, (255, 255, 255))
            rgb_img.paste(img, mask=img.split()[-1])
            img = rgb_img
        elif img.mode != 'RGB':
            img = img.convert('RGB')

        # Resize for model (224x224 for Teachable Machine)
        img_resized = img.resize((224, 224))
        img_array = np.array(img_resized).astype(np.float32) / 255.0
        img_array = np.expand_dims(img_array, axis=0)

        # Make prediction
        prediction = predict_fn(tf.constant(img_array))
        result_key = list(prediction.keys())[0]
        probabilities = prediction[result_key].numpy()[0]

        # Get prediction results
        predicted_class_idx = np.argmax(probabilities)
        confidence = probabilities[predicted_class_idx]
        predicted_class = class_names[predicted_class_idx]

        # Show visualization if requested
        if show_visualization:
            visualize_prediction(original_img, predicted_class,
                                confidence, probabilities)

    return {
```

```

        'filename': os.path.basename(image_path),
        'prediction': predicted_class,
        'confidence': confidence,
        'probabilities': probabilities,
        'path': image_path
    }

except Exception as e:
    print(f"❌ Error processing {image_path}: {e}")
    return None

def visualize_prediction(image, prediction, confidence, probabilities):
    """
    Visualize a single prediction
    """
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

    # Show image
    ax1.imshow(image)
    emoji = class_emojis.get(prediction, '📄')
    ax1.set_title(f"{emoji} {prediction.upper()} ({confidence:.1%})",
    fontsize=14, fontweight='bold')
    ax1.axis('off')

    # Show probability bars
    colors = ['green' if prediction == 'with signature' else 'red' for _
in class_names]
    bars = ax2.barh(class_names, probabilities, color=colors, alpha=0.7)
    ax2.set_xlabel('Confidence', fontsize=12)
    ax2.set_title('Prediction Confidence', fontsize=14)
    ax2.set_xlim(0, 1)

    # Add percentage labels
    for bar, prob in zip(bars, probabilities):
        ax2.text(bar.get_width() + 0.01, bar.get_y() + bar.get_height()/2,
        f'{prob:.1%}', va='center')

    plt.tight_layout()
    plt.show()

```

Step 5:

```
# ===== FOLDER PROCESSING FUNCTIONS =====  
def process_google_drive_folder(folder_path, show_individual_results=True,  
    visualize_samples=3):  
    """  
    Process all images in a Google Drive folder  
  
    Args:  
        folder_path: Path to folder in Google Drive (e.g.,  
        '/content/drive/MyDrive/signatures')  
        show_individual_results: Whether to print each result  
        visualize_samples: Number of sample visualizations to show  
    """  
    print("\n" + "="*60)  
    print("📁 SIGNATURE DETECTION FROM GOOGLE DRIVE FOLDER")  
    print("="*60)  
    print(f"📍 Folder: {folder_path}")  
  
    # Check if folder exists  
    if not os.path.exists(folder_path):  
        print(f"❌ Folder not found: {folder_path}")  
        print("💡 Tip: Make sure the path is correct. Example:")  
        print("    /content/drive/MyDrive/YourFolder")  
        return None  
  
    # Find all image files  
    image_extensions = ['*.jpg', '*.jpeg', '*.png', '*.gif', '*.bmp',  
        '*.JPG', '*.JPEG', '*.PNG']  
    image_files = []  
  
    for ext in image_extensions:  
        pattern = os.path.join(folder_path, ext)  
        image_files.extend(glob.glob(pattern))  
        # Also check subdirectories  
        pattern_sub = os.path.join(folder_path, '**', ext)  
        image_files.extend(glob.glob(pattern_sub, recursive=True))  
  
    # Remove duplicates  
    image_files = list(set(image_files))
```



```

if not image_files:
    print("❌ No image files found in the folder")
    return None

print(f"📷 Found {len(image_files)} images to process")
print("-"*60)

# Process each image
results = []
with_signature = []
without_signature = []
errors = []

print("\n🔄 Processing images...")
for i, image_path in enumerate(image_files, 1):
    # Progress indicator
    if i % 10 == 0 or i == len(image_files):
        print(f"    Progress: {i}/{len(image_files)}
({i/len(image_files)*100:.1f}%)")

    # Predict
    result = predict_image(image_path, show_visualization=False)

    if result:
        results.append(result)

    # Categorize
    if result['prediction'] == 'with signature':
        with_signature.append(result)
    else:
        without_signature.append(result)

    # Show individual result if requested
    if show_individual_results and i <= 10: # Show first 10
        emoji = class_emojis.get(result['prediction'], '📄')
        print(f"    {i}. {result['filename'][:30]:30} → {emoji}
{result['prediction']:20} ({result['confidence']:.1f}%)")
    else:
        errors.append(os.path.basename(image_path))

```

```

# Show sample visualizations
if visualize_samples > 0 and results:
    print(f"\n🖼️ Showing {min(visualize_samples, len(results))} sample
predictions:")
    for i in range(min(visualize_samples, len(results))):
        result = results[i]
        img = Image.open(result['path'])
        print(f"\nSample {i+1}: {result['filename']}")
        visualize_prediction(img, result['prediction'],
result['confidence'], result['probabilities'])

# Generate summary report
print("\n" + "="*60)
print("📊 PROCESSING COMPLETE - SUMMARY REPORT")
print("="*60)

print(f"\n✅ Overall Statistics:")
print(f"    Total images processed: {len(results)}")
print(f"    🖋️ With signature: {len(with_signature)}
({len(with_signature)/len(results)*100:.1f}%)")
print(f"    📄 Without signature: {len(without_signature)}
({len(without_signature)/len(results)*100:.1f}%)")
if errors:
    print(f"    ❌ Errors: {len(errors)}")

# Confidence statistics
if results:
    avg_confidence = np.mean([r['confidence'] for r in results])
    min_confidence = min(r['confidence'] for r in results)
    max_confidence = max(r['confidence'] for r in results)

    print(f"\n👉 Confidence Statistics:")
    print(f"    Average: {avg_confidence:.1%}")
    print(f"    Minimum: {min_confidence:.1%}")
    print(f"    Maximum: {max_confidence:.1%}")

# List files by category
print(f"\n📁 Files WITH Signature ({len(with_signature)}):")
if with_signature:

```

```

        for i, result in enumerate(with_signature[:10], 1): # Show first
10
            print(f"    {i}. {result['filename']}
({result['confidence']:.1%})")
            if len(with_signature) > 10:
                print(f"    ... and {len(with_signature)-10} more")

    print(f"\n📁 Files WITHOUT Signature ({len(without_signature)}):")
    if without_signature:
        for i, result in enumerate(without_signature[:10], 1): # Show
first 10
            print(f"    {i}. {result['filename']}
({result['confidence']:.1%})")
            if len(without_signature) > 10:
                print(f"    ... and {len(without_signature)-10} more")

# Files with low confidence (might need review)
low_confidence = [r for r in results if r['confidence'] < 0.8]
if low_confidence:
    print(f"\n⚠️ Files with Low Confidence (<80%) - May Need Review:")
    for result in low_confidence[:10]:
        emoji = class_emojis.get(result['prediction'], '📄')
        print(f"    {emoji} {result['filename']}:
{result['prediction']} ({result['confidence']:.1%})")

    return {
        'results': results,
        'with_signature': with_signature,
        'without_signature': without_signature,
        'errors': errors,
        'summary': {
            'total': len(results),
            'with_sig_count': len(with_signature),
            'without_sig_count': len(without_signature),
            'avg_confidence': avg_confidence if results else 0
        }
    }

def save_results_to_csv(results_dict,
output_path='/content/signature_detection_results.csv'):

```

```

"""
Save results to a CSV file
"""
if not results_dict or not results_dict['results']:
    print("No results to save")
    return

# Create DataFrame
df_data = []
for result in results_dict['results']:
    df_data.append({
        'Filename': result['filename'],
        'Prediction': result['prediction'],
        'Confidence': f"{result['confidence']:.2%}",
        'Has_Signature': 'Yes' if result['prediction'] == 'with
signature' else 'No',
        'Needs_Review': 'Yes' if result['confidence'] < 0.8 else 'No',
        'Full_Path': result['path']
    })

df = pd.DataFrame(df_data)

# Save to CSV
df.to_csv(output_path, index=False)
print(f"\n💾 Results saved to: {output_path}")

# Also save to Google Drive if mounted
drive_path =
f'/content/drive/MyDrive/signature_results_{datetime.now().strftime("%Y%m%
d_%H%M%S")}.csv'
try:
    df.to_csv(drive_path, index=False)
    print(f"💾 Also saved to Google Drive: {drive_path}")
except:
    pass

return df

def organize_files_by_signature(results_dict, source_folder,
output_base_folder='/content/organized_images'):

```

```

"""
Organize images into separate folders based on signature detection
"""
if not results_dict:
    print("No results to organize")
    return

# Create output folders
with_sig_folder = os.path.join(output_base_folder, 'with_signature')
without_sig_folder = os.path.join(output_base_folder,
'without_signature')
review_folder = os.path.join(output_base_folder, 'needs_review')

os.makedirs(with_sig_folder, exist_ok=True)
os.makedirs(without_sig_folder, exist_ok=True)
os.makedirs(review_folder, exist_ok=True)

print(f"\n📁 Organizing files into folders...")

# Copy files to appropriate folders
for result in results_dict['results']:
    source = result['path']
    filename = result['filename']

    if result['confidence'] < 0.8:
        destination = os.path.join(review_folder, filename)
    elif result['prediction'] == 'with signature':
        destination = os.path.join(with_sig_folder, filename)
    else:
        destination = os.path.join(without_sig_folder, filename)

    try:
        shutil.copy2(source, destination)
    except Exception as e:
        print(f"❌ Error copying {filename}: {e}")

print(f"✅ Files organized in: {output_base_folder}")
print(f"📁 with_signature/      :
{len(results_dict['with_signature'])} files")

```

```
print(f"📁 without_signature/ :  
{len(results_dict['without_signature'])} files")  
print(f"📁 needs_review/ : {len([r for r in  
results_dict['results'] if r['confidence'] < 0.8])} files")
```

Step 6:

```
# ===== MAIN EXECUTION =====

def main():
    """
    Main function to run signature detection
    """
    print("\n" + "="*60)
    print("🔒 SIGNATURE DETECTION SYSTEM")
    print("="*60)

    print("\n📁 Enter the Google Drive folder path to process:")
    print("    Example: /content/drive/MyDrive/signatures")
    print("    Or press Enter to use default: /content/drive/MyDrive")

    folder_path = input("\nFolder path: ").strip()
    if not folder_path:
        folder_path = '/content/drive/MyDrive'

    # Process the folder
    results = process_google_drive_folder(
        folder_path=folder_path,
        show_individual_results=True,
        visualize_samples=3
    )

    if results:
        # Ask if user wants to save results
        print("\n💾 Do you want to save the results to CSV? (y/n)")
        if input().strip().lower() in ['y', 'yes']:
            df = save_results_to_csv(results)

        # Ask if user wants to organize files
        print("\n📁 Do you want to organize files into folders by signature status? (y/n)")
        if input().strip().lower() in ['y', 'yes']:
            organize_files_by_signature(results, folder_path)

    print("\n✅ Processing complete!")
    return results
```

```
# Run the main function
if __name__ == "__main__":
    results = main()
```

Step 7: Data Frame validation

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
import warnings
warnings.filterwarnings('ignore')

# Set style for better visualizations
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

# =====
# STEP 1: LOAD YOUR DATA
# =====

csv_path = '/content/signature_detection_results.csv' # Update with your
actual path
df = pd.read_csv(csv_path)

# Convert Confidence to numeric if needed
if 'Confidence' in df.columns and df['Confidence'].dtype == 'object':
    df['Confidence_Numeric'] =
df['Confidence'].str.rstrip('%').astype('float') / 100.0
elif 'Confidence' in df.columns:
    df['Confidence_Numeric'] = df['Confidence']
```


Step 8:

```
# =====
# STEP 2: VALIDATE THE DATAFRAME
# =====

def validate_dataframe(df):
    """
    Validate that DataFrame has required columns for EDA
    """
    required_columns = ['Filename', 'Prediction', 'Has_Signature',
                        'Confidence_Numeric']
    missing_columns = [col for col in required_columns if col not in
                        df.columns]

    if missing_columns:
        print(f"❌ Missing required columns: {missing_columns}")
        print(f"    Available columns: {list(df.columns)}")
        return False

    if len(df) == 0:
        print("❌ DataFrame is empty")
        return False

    if not pd.api.types.is_numeric_dtype(df['Confidence_Numeric']):
        print("❌ Confidence_Numeric must be numeric")
        return False

    print(f"✅ DataFrame validation passed")
    print(f"    - Records: {len(df)}")
    print(f"    - Columns: {list(df.columns)}")
    return True

# Validate the data
if not validate_dataframe(df):
    print("⚠️ Please fix data issues before proceeding")
else:
    print(f"\n🎉 Data is ready for analysis!")
```

EDA Development

Step 1:

```
# =====  
# EDA FOR SMALL DATASETS  
# =====  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
from datetime import datetime  
import warnings  
warnings.filterwarnings('ignore')  
  
# Set style for better visualizations  
plt.style.use('seaborn-v0_8-darkgrid')  
sns.set_palette("husl")
```

Step2:

```
# =====  
# BLOCK 1: PERFORMANCE METRICS (FIXED)  
# =====  
  
def calculate_performance_metrics_fixed(df):  
    """  
    Calculate and visualize model performance metrics - FIXED for small  
    datasets  
    """  
    print("\n🎯 MODEL PERFORMANCE METRICS")  
    print("="*60)  
  
    # Check if we have enough data  
    min_samples_kde = 2  
    has_enough_data = all(  
        len(df[df['Has_Signature'] == status]) >= min_samples_kde  
        for status in df['Has_Signature'].unique()  
    )
```

```

# Adjust subplot layout based on data availability
if has_enough_data:
    fig, axes = plt.subplots(2, 2, figsize=(15, 10))
else:
    fig, axes_temp = plt.subplots(2, 2, figsize=(15, 10))
    axes = axes_temp.flatten()

fig.suptitle('📊 Model Performance Analysis', fontsize=16,
fontweight='bold')

# 1. Confidence Score Distribution (Bar chart for small data, KDE for
large)
ax1 = axes[0] if not has_enough_data else axes[0, 0]

if len(df) < 20 or not has_enough_data:
    # Use bar chart for small datasets
    for sig_status in df['Has_Signature'].unique():
        subset = df[df['Has_Signature'] ==
sig_status]['Confidence_Numeric']
        x_pos = np.arange(len(subset))
        colors = ['green' if sig_status == 'Yes' else 'red']
        ax1.bar(x_pos + (0.4 if sig_status == 'Yes' else 0),
                subset.values,
                width=0.4,
                label=f'{sig_status} (n={len(subset)})',
                color=colors[0],
                alpha=0.7)

        ax1.set_xlabel('Document Index', fontsize=11)
        ax1.set_ylabel('Confidence Score', fontsize=11)
        ax1.set_title('Confidence Scores by Category', fontsize=12,
fontweight='bold')
        ax1.legend()
        ax1.grid(True, alpha=0.3)
        ax1.set_ylim(0, 1.1)
else:
    # Use KDE for larger datasets
    try:
        for sig_status in df['Has_Signature'].unique():

```

```

subset = df[df['Has_Signature'] ==
sig_status]['Confidence_Numeric']
    if len(subset) >= min_samples_kde:
        subset.plot.kde(ax=ax1, label=sig_status, linewidth=2)

ax1.set_xlabel('Confidence Score', fontsize=11)
ax1.set_ylabel('Density', fontsize=11)
ax1.set_title('Confidence Score Density by Category',
fontsize=12, fontweight='bold')
ax1.legend()
ax1.grid(True, alpha=0.3)
ax1.set_xlim(0, 1)
except:
    # Fallback to histogram if KDE fails
    ax1.hist([df[df['Has_Signature'] == s]['Confidence_Numeric']
        for s in df['Has_Signature'].unique()],
        label=df['Has_Signature'].unique(),
        alpha=0.7)
    ax1.set_xlabel('Confidence Score', fontsize=11)
    ax1.set_ylabel('Count', fontsize=11)
    ax1.set_title('Confidence Distribution', fontsize=12,
fontweight='bold')
    ax1.legend()

# 2. Scatter Plot for Small Data / Decision Boundary for Large
ax2 = axes[1] if not has_enough_data else axes[0, 1]

if len(df) < 30:
    # Simple scatter plot for small datasets
    colors = ['green' if x == 'Yes' else 'red' for x in
df['Has_Signature']]
    sizes = df['Confidence_Numeric'] * 200 # Scale for visibility

    ax2.scatter(range(len(df)), df['Confidence_Numeric'],
        c=colors, s=sizes, alpha=0.6, edgecolors='black')
    ax2.axhline(0.8, color='orange', linestyle='--', label='Review
Threshold (80%)')
    ax2.set_xlabel('Document Index', fontsize=11)
    ax2.set_ylabel('Confidence Score', fontsize=11)

```

```

        ax2.set_title('Individual Document Confidence', fontsize=12,
fontweight='bold')
        ax2.legend()
        ax2.grid(True, alpha=0.3)

        # Add document labels
        for i, row in df.iterrows():
            ax2.annotate(row['Filename'][:10],
                        (i, row['Confidence_Numeric']),
                        fontsize=8, rotation=45)
    else:
        # Decision boundary analysis for larger datasets
        bins = np.linspace(0, 1, min(21, len(df)//2))

        for sig_status, color in zip(['Yes', 'No'], ['green', 'red']):
            if sig_status in df['Has_Signature'].values:
                subset = df[df['Has_Signature'] ==
sig_status]['Confidence_Numeric']
                counts, edges = np.histogram(subset, bins=bins)
                centers = (edges[:-1] + edges[1:]) / 2
                ax2.plot(centers, counts, marker='o',
label=f'{sig_status}',
                        color=color, linewidth=2)

            ax2.axvline(0.8, color='orange', linestyle='--', label='Review
Threshold (80%)')
            ax2.set_xlabel('Confidence Score', fontsize=11)
            ax2.set_ylabel('Count', fontsize=11)
            ax2.set_title('Decision Boundary Analysis', fontsize=12,
fontweight='bold')
            ax2.legend()
            ax2.grid(True, alpha=0.3)

    # 3. Summary Statistics Table
    ax3 = axes[2] if not has_enough_data else axes[1, 0]

    # Create detailed statistics table
    stats_data = []

    # Overall stats

```

```

stats_data.append([
    'OVERALL',
    len(df),
    f"{df['Confidence_Numeric'].mean():.1%}",
    f"{df['Confidence_Numeric'].median():.1%}",
    f"{df['Confidence_Numeric'].std():.1%}"
])

# Per category stats
for category in sorted(df['Has_Signature'].unique()):
    cat_data = df[df['Has_Signature'] == category]
    stats_data.append([
        f'{category}',
        len(cat_data),
        f"{cat_data['Confidence_Numeric'].mean():.1%}",
        f"{cat_data['Confidence_Numeric'].median():.1%}",
        f"{cat_data['Confidence_Numeric'].std():.1%}" if len(cat_data)
> 1 else "N/A"
    ])

# Hide axes
ax3.axis('tight')
ax3.axis('off')

# Create table
table = ax3.table(cellText=stats_data,
                  colLabels=['Category', 'Count', 'Mean', 'Median',
'Std Dev'],
                  cellLoc='center',
                  loc='center',
                  colWidths=[0.25, 0.15, 0.2, 0.2, 0.2])

table.auto_set_font_size(False)
table.set_fontsize(11)
table.scale(1.2, 2)

# Style the table
for i in range(len(stats_data) + 1):
    for j in range(5):
        if i == 0: # Header

```

```

        table[(i, j)].set_facecolor('#3498db')
        table[(i, j)].set_text_props(weight='bold', color='white')
    elif stats_data[i-1][0] == 'OVERALL': # Overall row
        table[(i, j)].set_facecolor('#95a5a6')
        table[(i, j)].set_text_props(weight='bold')
    else:
        table[(i, j)].set_facecolor('#ecf0f1')

ax3.set_title('Statistical Summary', fontsize=12, fontweight='bold',
pad=20)

# 4. Confidence Range Chart
ax4 = axes[3] if not has_enough_data else axes[1, 1]

# Show confidence ranges
categories = []
ranges = []
means = []

for category in sorted(df['Has_Signature'].unique()):
    cat_data = df[df['Has_Signature'] ==
category]['Confidence_Numeric']
    categories.append(f'{category}\n(n={len(cat_data)})')
    if len(cat_data) > 0:
        ranges.append([cat_data.min(), cat_data.max()])
        means.append(cat_data.mean())
    else:
        ranges.append([0, 0])
        means.append(0)

# Plot ranges
x_pos = np.arange(len(categories))
for i, (cat, range_vals, mean_val) in enumerate(zip(categories,
ranges, means)):
    ax4.plot([i, i], range_vals, 'b-', linewidth=3, alpha=0.5)
    ax4.plot(i, mean_val, 'ro', markersize=10)
    ax4.plot(i, range_vals[0], 'v', markersize=8, color='green')
    ax4.plot(i, range_vals[1], '^', markersize=8, color='red')

ax4.set_xticks(x_pos)

```

```

ax4.set_xticklabels(categories)
ax4.set_ylabel('Confidence Score', fontsize=11)
ax4.set_title('Confidence Ranges by Category', fontsize=12,
fontweight='bold')
ax4.grid(True, alpha=0.3, axis='y')
ax4.set_ylim(0, 1.1)

# Add legend
ax4.plot([], [], 'ro', markersize=8, label='Mean')
ax4.plot([], [], 'v', markersize=8, color='green', label='Min')
ax4.plot([], [], '^', markersize=8, color='red', label='Max')
ax4.legend(loc='upper right')

plt.tight_layout()
plt.show()

# Performance Summary
print("\n Performance Summary:")
print("-"*40)

if len(df) > 0:
    high_conf = df['Confidence_Numeric'] >= 0.9
    medium_conf = (df['Confidence_Numeric'] >= 0.8) &
(df['Confidence_Numeric'] < 0.9)
    low_conf = df['Confidence_Numeric'] < 0.8

    print(f"High Confidence (>90%): {high_conf.sum()} documents
({high_conf.mean():.1%})")
    print(f"Medium Confidence (80-90%): {medium_conf.sum()} documents
({medium_conf.mean():.1%})")
    print(f"Low Confidence (<80%): {low_conf.sum()} documents
({low_conf.mean():.1%})")

    print(f"\n🎯 Model Reliability:")
    print(f"    Documents above 80% confidence:
{(df['Confidence_Numeric'] >= 0.8).sum()}/{len(df)}
({(df['Confidence_Numeric'] >= 0.8).mean():.1%})")
    print(f"    Documents above 90% confidence:
{(df['Confidence_Numeric'] >= 0.9).sum()}/{len(df)}
({(df['Confidence_Numeric'] >= 0.9).mean():.1%})")

```



```

    print(f"    Documents above 95% confidence:
{(df['Confidence_Numeric'] >= 0.95).sum()}/{len(df)}
{(df['Confidence_Numeric'] >= 0.95).mean():.1%}")

    # Add specific insights for small datasets
    if len(df) < 10:
        print(f"\n📝 Small Dataset Insights:")
        print(f"    Total samples: {len(df)} (consider collecting more
data)")

        print(f"    Signature ratio: {(df['Has_Signature'] ==
'Yes').sum()}/{(df['Has_Signature'] == 'No').sum()}")

        # List all files with their status
        print(f"\n📁 All Documents:")
        for _, row in df.iterrows():
            status_emoji = "👤" if row['Has_Signature'] == 'Yes' else
"📄"
            review_flag = "⚠️" if row['Needs_Review'] == 'Yes' else
"✅"

            print(f"    {status_emoji} {row['Filename'][:30]:30} -
{row['Confidence_Numeric']:.1%} {review_flag}")

# Test with your data
calculate_performance_metrics_fixed(df)

```

Step 3:

```

# =====
# BLOCK 2: SIMPLE SUMMARY FOR SMALL DATASETS
# =====

def simple_summary_visualization(df):
    """
    Create simple, clear visualizations for small datasets
    """
    print("\n📊 SIMPLE SUMMARY VISUALIZATION")
    print("="*60)

    fig, axes = plt.subplots(1, 3, figsize=(15, 5))

```

```

fig.suptitle(f'Signature Detection Summary (n={len(df)}),
fontsize=14, fontweight='bold')

# 1. Individual document bars
ax1 = axes[0]
colors = ['green' if x == 'Yes' else 'red' for x in
df['Has_Signature']]
bars = ax1.bar(range(len(df)), df['Confidence_Numeric'], color=colors,
alpha=0.7, edgecolor='black')
ax1.axhline(0.8, color='orange', linestyle='--', label='Review
Threshold', linewidth=2)
ax1.set_xlabel('Document', fontsize=11)
ax1.set_ylabel('Confidence Score', fontsize=11)
ax1.set_title('Document Confidence Scores', fontsize=12)
ax1.set_ylim(0, 1.1)
ax1.legend()
ax1.grid(True, alpha=0.3, axis='y')

# Add value labels
for bar, conf in zip(bars, df['Confidence_Numeric']):
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
            f'{conf:.0%}', ha='center', fontsize=10,
fontweight='bold')

# 2. Count summary
ax2 = axes[1]
counts = df['Has_Signature'].value_counts()
colors_pie = ['#2ecc71' if 'Yes' in counts.index else '#e74c3c',
            '#e74c3c' if 'No' in counts.index else '#2ecc71']

wedges, texts, autotexts = ax2.pie(counts.values,
                                labels=[f'{label}\n({count})' for
label, count in zip(counts.index, counts.values)],
                                colors=colors_pie[:len(counts)],
                                autopct='%1.0f%%',
                                startangle=90)
ax2.set_title('Signature Distribution', fontsize=12)

# 3. Document list with details
ax3 = axes[2]

```

```

ax3.axis('off')

# Create document details table
doc_data = []
for _, row in df.iterrows():
    emoji = "👤" if row['Has_Signature'] == 'Yes' else "📄"
    review = "⚠️ Review" if row['Needs_Review'] == 'Yes' else "✅ OK"
    filename_short = row['Filename'][:20] + '...' if
len(row['Filename']) > 20 else row['Filename']
    doc_data.append([
        filename_short,
        f"{emoji} {row['Prediction'][:10]}",
        f"{row['Confidence_Numeric']:.0%}",
        review
    ])

table = ax3.table(cellText=doc_data,
                  colLabels=['File', 'Status', 'Conf.', 'Review'],
                  cellLoc='left',
                  loc='center',
                  colWidths=[0.35, 0.25, 0.15, 0.25])

table.auto_set_font_size(False)
table.set_fontsize(9)
table.scale(1, 1.5)

# Style the table
for i in range(len(doc_data) + 1):
    for j in range(4):
        if i == 0: # Header
            table[(i, j)].set_facecolor('#3498db')
            table[(i, j)].set_text_props(weight='bold', color='white')
        else:
            if doc_data[i-1][3] == "⚠️ Review":
                table[(i, j)].set_facecolor('#ffe6e6')
            else:
                table[(i, j)].set_facecolor('#e6ffe6')

ax3.set_title('Document Details', fontsize=12, pad=20)

```

```

plt.tight_layout()
plt.show()

# Print insights
print("\n💡 Key Insights for Small Dataset:")
print("-"*40)
print(f"✓ Processed {len(df)} documents")
print(f"✓ {(df['Has_Signature'] == 'Yes').sum()} with signatures,
{(df['Has_Signature'] == 'No').sum()} without")
print(f"✓ Average confidence: {df['Confidence_Numeric'].mean():.1%}")

if df['Needs_Review'].any():
    review_files = df[df['Needs_Review'] ==
'Yes']['Filename'].tolist()
    print(f"\n⚠ Files needing review:")
    for f in review_files:
        print(f"    - {f}")

print(f"\n📋 Recommendations:")
if len(df) < 10:
    print("    1. Consider processing more documents for better
statistical analysis")
    if (df['Confidence_Numeric'] < 0.8).any():
        print("    2. Review low-confidence predictions manually")
    if df['Confidence_Numeric'].mean() > 0.9:
        print("    3. Model showing good confidence - results are
reliable")

# Run simple summary
simple_summary_visualization(df)

```

Step 4:

```
# =====  
# BLOCK 3: DETAILED FILE REVIEW  
# =====  
  
def detailed_file_review(df):  
    """  
    Create a detailed review of each file for small datasets  
    """  
    print("\n📋 DETAILED FILE REVIEW")  
    print("="*60)  
  
    for idx, row in df.iterrows():  
        print(f"\n{idx+1}. {row['Filename']}")  
        print("-"*40)  
  
        # Determine emoji and color  
        sig_emoji = "👉" if row['Has_Signature'] == 'Yes' else "📄"  
        conf_level = "High" if row['Confidence_Numeric'] >= 0.9 else  
"Medium" if row['Confidence_Numeric'] >= 0.8 else "Low"  
  
        print(f"    Status: {sig_emoji} {row['Prediction']}")  
        print(f"    Confidence: {row['Confidence_Numeric']:.1%}  
({conf_level})")  
        print(f"    Needs Review: {'⚠️ Yes' if row['Needs_Review'] == 'Yes'  
else '✅ No'}")  
  
        # Add recommendation  
        if row['Needs_Review'] == 'Yes':  
            print(f"    📝 Recommendation: Manual verification required")  
        elif row['Confidence_Numeric'] >= 0.95:  
            print(f"    📝 Recommendation: High confidence - result  
reliable")  
        else:  
            print(f"    📝 Recommendation: Result acceptable, periodic  
validation suggested")  
  
    # Summary statistics  
    print("\n" + "="*60)  
    print("SUMMARY STATISTICS")
```

```

print("="*60)

print(f"\n📊 Overall Performance:")
print(f"    Total Files: {len(df)}")
print(f"    Average Confidence: {df['Confidence_Numeric'].mean():.1%}")
print(f"    Confidence Range: {df['Confidence_Numeric'].min():.1%} - {df['Confidence_Numeric'].max():.1%}")

print(f"\n📁 File Categories:")
for status in df['Has_Signature'].unique():
    count = (df['Has_Signature'] == status).sum()
    pct = count / len(df) * 100
    avg_conf = df[df['Has_Signature'] == status]['Confidence_Numeric'].mean()
    print(f"    {status}: {count} files ({pct:.0f}%) - Avg confidence: {avg_conf:.1%}")

# Run detailed review
detailed_file_review(df)

```